

Decoder-Only Transformer for Language Modeling

Architecture summary, math, and implementation notes — built from a Word2Vec-based LM

1. Overview

The pipeline builds a small autoregressive (decoder-only) Transformer language model: Word2Vec-pretrained token embeddings + sinusoidal positional encodings feed into a stack of self-attention blocks under a causal mask, followed by a linear language-modeling head that predicts the next token. Training minimizes cross-entropy on next-token prediction; evaluation uses perplexity.

2. Input Embeddings

Token embedding (Word2Vec, pretrained)

Token lookup is a one-hot times matrix operation. With a pretrained embedding matrix E (shape $\text{vocab_size} \times d_{\text{model}}$), d_{model} is *derived* from E 's second dimension, not chosen independently — the positional encoding must match it exactly for elementwise addition.

```
tok_emb(x_t) = E[x_t]      (× sqrt(d_model) in the from-scratch version)
```

The $\sqrt{d_{\text{model}}}$ scale exists because Xavier-initialized learned embeddings have variance $\approx 1/d$, so scaling balances their magnitude against the positional encoding. Pretrained Word2Vec vectors already have their own empirical norm distribution, so this scaling assumption should be checked before reusing it verbatim.

Sinusoidal positional encoding

```
PE(pos, 2i) = sin( pos / 10000^(2i/d) )
PE(pos, 2i+1) = cos( pos / 10000^(2i/d) )
```

Each dimension pair oscillates at a different frequency — low index = fast-changing, high index = slow-changing — giving each position a unique, bounded fingerprint. Because $\sin/\cos$ angle-addition identities make $\text{PE}(\text{pos}+k)$ a linear function of $\text{PE}(\text{pos})$, attention can in principle learn relative-position relationships.

```
x_t = tok_emb(x_t) + PE(t)  (elementwise addition, so shapes must match)
```

3. Multi-Head Self-Attention

The only component where tokens exchange information; everything else acts per-token independently. Q, K, V are linear projections of the input, split into num_heads parallel heads of dimension $d_k = d_{\text{model}} / \text{num_heads}$.

```
Q = X W_Q, K = X W_K, V = X W_V
scores = (Q K^T) / sqrt(d_k)
A = softmax(scores + mask)
O = A V, then concat heads and project with W_O
```

The $1/\sqrt{d_k}$ scale prevents dot products from growing large in magnitude as d_k increases, which would otherwise push softmax into saturated (near one-hot, vanishing-gradient) regions.

4. Causal Masking — What Makes It Decoder-Only

For language modeling, position t must not attend to positions $> t$ (no peeking at the answer). This is enforced with a lower-triangular mask applied before softmax:

```
mask[i, j] = 1 if j ≤ i else 0
scores = scores.masked_fill(mask == 0, -inf)
```

Bug caught in the original notebook: no causal mask was applied anywhere, and the encoder layer's `__init__` never called `super().__init__()`, silently breaking submodule registration. Without the mask, the stack is a bidirectional encoder, not a valid autoregressive LM — next-token prediction becomes trivial information leakage rather than a genuine modeling task.

5. Block Internals: Residual, LayerNorm, Feed-Forward

Residual connection

```
y = x + Sublayer(x)
```

Gives gradients a direct identity path through depth ($\partial y / \partial x = I + \partial \text{Sublayer} / \partial x$), which is what makes deep stacks trainable at all. Each layer only needs to learn a correction to add, not reconstruct the representation from scratch.

Layer normalization

$$\text{LN}(x) = \gamma \cdot (x - \mu) / \sqrt{\sigma^2 + \epsilon} + \beta$$

Normalizes across the feature dimension per token (independent of batch), stabilizing activation scale as residual sums accumulate across layers. γ, β are learned, so normalization is not a hard constraint.

Position-wise feed-forward network

$$\text{FFN}(x) = \text{ReLU}(x W_1 + b_1) W_2 + b_2 \quad (\text{d}_{\text{ff}} \text{ typically } 4 \times \text{d}_{\text{model}})$$

Applied identically and independently to every position. Attention only produces linear combinations in V -space — it cannot express nonlinear per-token feature interactions on its own. The FFN's nonlinearity is where most per-token representational capacity actually lives.

Block order: $x \rightarrow \text{self-attn} \rightarrow \text{residual} + \text{LN} \rightarrow \text{FFN} \rightarrow \text{residual} + \text{LN}$. Stacking N of these blocks gives $x_{\text{final}} \approx x_0 + \text{sum of per-layer corrections}$.

6. Language Modeling Head, Loss, and Perplexity

```
logits = h W_lm^T    (h = final hidden state at position t, shape d_model)
p(w_t | w_{<t}) = softmax(logits)
L = - log p(w_t^true | w_{<t})
perplexity = exp( E[L] )
```

Perplexity is the effective average branching factor the model is uncertain between at each step — lower is better, and a perfectly confident correct model has perplexity 1.

Weight tying pitfall: tying `lm_head.weight` to a `Word2Vec` embedding loaded with `freeze=True` makes the LM head untrainable too, since `requires_grad=False` propagates through the shared tensor. Only tie weights when the embedding is trainable.

7. Training & Evaluation Mechanics

- Next-token shift: `input = ids[:, :-1]`, `target = ids[:, 1:]` (LM objective needs shifted pairs; the dataset only returns single padded sequences, so the shift is done in the loop).
- `CrossEntropyLoss(ignore_index=PAD_IDX)` — otherwise pad tokens corrupt both loss and perplexity.
- Combined mask = causal triangular mask AND padding mask, so the model neither sees the future nor attends into pad positions.
- Perplexity is computed as $\exp(\text{total loss over non-pad tokens} / \text{total non-pad token count})$, aggregated across the whole loader — not averaged per-batch, to avoid bias from unequal batch token counts.

8. Component Summary

Component	Role	Mixes tokens?
Input Embeddings	Token identity + position	No
Multi-Head Attention	Gathers context from other (allowed) tokens	Yes
Causal mask	Restricts attention to positions $\leq t$	—
Residual + LayerNorm	Trainability / stability across depth	No
Feed-Forward Network	Nonlinear per-token transformation	No
LM Head + Softmax	Hidden state $\rightarrow$ vocabulary distribution	No